

COMP2271: Computer Graphics

Part 2: 2D and 3D Transformations

Dr. Frederick Li

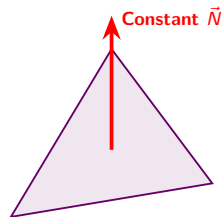
Department of Computer Science
Durham University

Revision: The Polygon Model (Part 1)

Mathematical Stability and Lighting Calculation

Why do we build worlds out of Triangles?

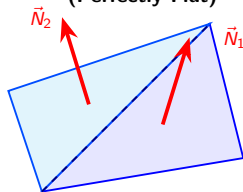
- **Guaranteed Flatness:** A triangle is the simplest polygon. Mathematically, any three points in space define a perfectly flat (coplanar) plane.
- **Well-Defined Normals:** Because it is flat, every point on a triangle's surface shares exactly one constant normal vector ($\vec{N}$).
- **Precise Lighting:** This constant normal makes calculating how light bounces off the surface mathematically simple and highly predictable.



Triangle
(Perfectly Flat)

The Problem with Quads (4 Vertices)

- Unlike triangles, a shape with 4 vertices is **not guaranteed to be flat**. If just one vertex is out of alignment, the shape becomes **bent**.
- A bent quad essentially faces more than one direction. It loses its single, constant normal vector, making lighting calculations ambiguous and problematic for the graphics rendering.



Bent Quad

Breaks into 2 Triangles
($\vec{N}_1 \neq \vec{N}_2$)

Revision: The Polygon Model (Part 2)

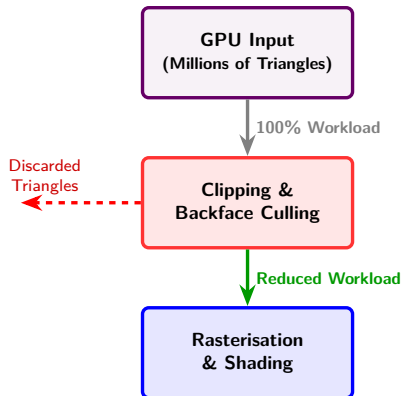
GPU Hardware and Pipeline Optimisation

Designed for Triangles

- Modern GPUs are fundamentally hardwired to accept and process triangles. The entire **Rasterisation-based Rendering Pipeline** is built around this specific geometry.

The Strategy of Reduction

- The Bottleneck:** Rasterisation (converting shapes to pixels) and Shading (calculating colour/light per pixel) are the most expensive stages of rendering.
- The Solution:** We must reduce the workload *before* we reach those stages.
- Pipeline Filters:** Hardware stages like **Clipping** (removing off-screen triangles) and **Backface Culling** (removing rear-facing triangles) act as ruthless filters.



The Frozen World Problem

Imagine a world without movement...

Picture a video game where you have built a magnificent city.

- You have modelled skyscrapers, cars, and characters.
- You hit "Run".
- The scene renders beautifully... **but nothing moves.**

The Frozen World Problem

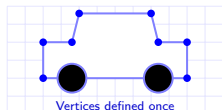
Imagine a world without movement...

Picture a video game where you have built a magnificent city.

- You have modelled skyscrapers, cars, and characters.
- You hit "Run".
- The scene renders beautifully... **but nothing moves.**

The cars are glued to the road. The characters are statues. To move a car forward, you would have to manually erase it and redraw it one metre ahead, vertex by vertex, millions of times.

1. The Static Data



The Transformation Engine

Matrix Math
(Translate, Rotate)

2. The Living World



The Frozen World Problem

Imagine a world without movement...

Picture a video game where you have built a magnificent city.

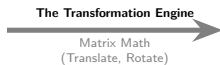
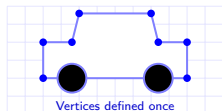
- You have modelled skyscrapers, cars, and characters.
- You hit "Run".
- The scene renders beautifully... **but nothing moves.**

The cars are glued to the road. The characters are statues. To move a car forward, you would have to manually erase it and redraw it one metre ahead, vertex by vertex, millions of times.

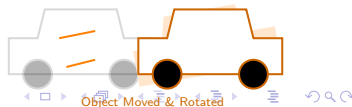
The Challenge

We need a mathematical engine that can take a static statue and turn it into a living, breathing world without rebuilding it from scratch every millisecond. That engine is **Transformation**.

1. The Static Data



2. The Living World



Lecture Outline

- 1 Introduction to 2D Transformations
- 2 Unifying with Matrices
- 3 3D Transformations & The MVP Pipeline
- 4 Practical Example: three.js
- 5 Appendix: Manual Composition

Why Do We Need Transformations?

From Static Scenes to Dynamic Worlds

So far, we can render objects, but they are static, i.e., their vertices have fixed positions.

Why Do We Need Transformations?

From Static Scenes to Dynamic Worlds

So far, we can render objects, but they are static, i.e., their vertices have fixed positions.

- To create dynamic and interactive scenes, we need to **move**, **rotate**, and **resize** objects.
- These operations are collectively known as **transformations**.

Why Do We Need Transformations?

From Static Scenes to Dynamic Worlds

So far, we can render objects, but they are static, i.e., their vertices have fixed positions.

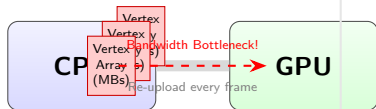
- To create dynamic and interactive scenes, we need to **move**, **rotate**, and **resize** objects.
- These operations are collectively known as **transformations**.

The Naive Approach (and why it is bad)

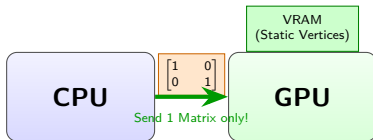
We could change vertex positions on the CPU and re-upload the data every frame. This is extremely inefficient!

- **High CPU Cost:** Wastes CPU cycles on repetitive calculations.
- **Slow Data Transfer:** The CPU-GPU bus is a bottleneck.

1. The Naive Way (Slow)



2. The Matrix Way (Fast)



What is Transformation?

The Mathematical Engine of Computer Graphics

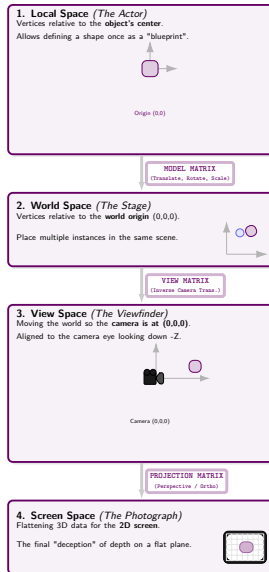
Definition:

In Computer Graphics, a **transformation** is a mathematical operation that changes the **position**, **rotation**, or **scale** of geometric data.

Why is it crucial?

3D models are stored as static numbers (e.g., a cube centred at $0, 0, 0$). To create a dynamic world, we must move this data through different "Coordinate Spaces":

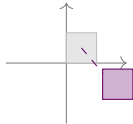
1. **Model:** Shaping the object.
2. **World:** Placing it in the scene.
3. **View:** Aligning it to the camera.
4. **Screen:** Flattening it to pixels.



The Three Fundamental 2D Transformations

Translation

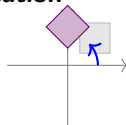
(Moving)



$$P' = P + T$$

Rotation

(Turning)

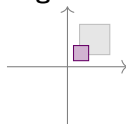


$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

Scaling

(Resizing)



$$P' = P \cdot S$$

Transformation 1: Translation

- **Concept:** Shifting an object from one location to another.
- **Defined by:** A translation vector $T = (t_x, t_y)$.
- **Operation:** Simple vector addition.

$$P' = P + T = (x + t_x, y + t_y)$$

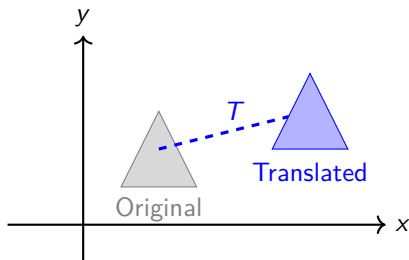


Figure: Every vertex is shifted by the same vector T .

Transformation 2: Rotation

- **Concept:** Turning an object around a pivot point (typically the origin).
- **Defined by:** An angle θ .
- **Operation:** Trigonometric formulae.

$$x' = x \cos(\theta) - y \sin(\theta)$$

$$y' = x \sin(\theta) + y \cos(\theta)$$

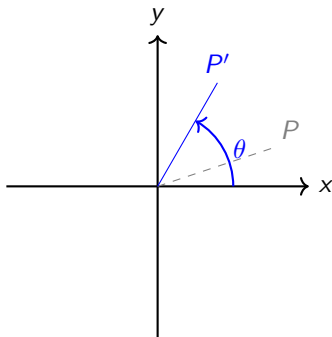


Figure: A point rotated by θ around the origin.

Transformation 3: Scaling

- **Concept:** Resizing an object relative to the origin.
- **Defined by:** A scaling vector $S = (s_x, s_y)$.
- **Operation:** Component-wise multiplication.

$$x' = x \cdot s_x \quad , \quad y' = y \cdot s_y$$

- If $s_x = s_y$, scaling is **uniform** (preserves aspect ratio).
- If $s_x \neq s_y$, scaling is **non-uniform** (stretches/squashes).

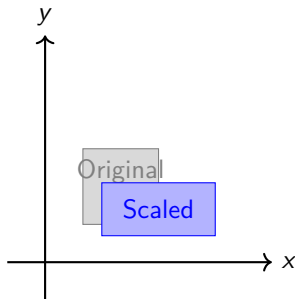


Figure: Non-uniform scaling with $S = (1.5, 0.7)$.

The Problem: A Lack of Unity

How can we treat all transformations the same way?

Currently, we use three fundamentally different mathematical operations to move objects:

- **Translation** $\rightarrow$ **Addition** ($P + T$)

The "Fragmented" Reality



The Problem: A Lack of Unity

How can we treat all transformations the same way?

Currently, we use three fundamentally different mathematical operations to move objects:

- **Translation** $\rightarrow$ **Addition** ($P + T$)
- **Rotation** $\rightarrow$ **Trigonometry**
($x \cos \theta - y \sin \theta$)

The "Fragmented" Reality



The solution is **Linear Algebra** and **Matrices**!

The Problem: A Lack of Unity

How can we treat all transformations the same way?

Currently, we use three fundamentally different mathematical operations to move objects:

- **Translation** $\rightarrow$ **Addition** ($P + T$)
- **Rotation** $\rightarrow$ **Trigonometry**
($x \cos \theta - y \sin \theta$)
- **Scaling** $\rightarrow$ **Multiplication** ($P \cdot S$)

The "Fragmented" Reality



The solution is **Linear Algebra** and **Matrices**!

The Goal: Represent every transformation as a single, consistent matrix-vector multiplication:

$$P' = M_{transform} \cdot P$$

The Problem: A Lack of Unity

How can we treat all transformations the same way?

Currently, we use three fundamentally different mathematical operations to move objects:

- **Translation** $\rightarrow$ **Addition** ($P + T$)
- **Rotation** $\rightarrow$ **Trigonometry**
($x \cos \theta - y \sin \theta$)
- **Scaling** $\rightarrow$ **Multiplication** ($P \cdot S$)

The "Fragmented" Reality



The Inconvenience

This "fragmented" approach is inefficient for a graphics pipeline. We cannot easily "chain" these operations together if they use different rules.

The solution is **Linear Algebra** and **Matrices**!

The Goal: Represent every transformation as a single, consistent matrix-vector multiplication:

$$P' = M_{transform} \cdot P$$

The Problem: A Lack of Unity

How can we treat all transformations the same way?

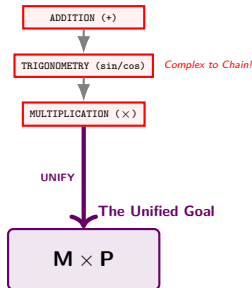
Currently, we use three fundamentally different mathematical operations to move objects:

- **Translation** $\rightarrow$ **Addition** ($P + T$)
- **Rotation** $\rightarrow$ **Trigonometry**
($x \cos \theta - y \sin \theta$)
- **Scaling** $\rightarrow$ **Multiplication** ($P \cdot S$)

The Inconvenience

This "fragmented" approach is inefficient for a graphics pipeline. We cannot easily "chain" these operations together if they use different rules.

The "Fragmented" Reality



The solution is **Linear Algebra** and **Matrices**!

The Goal: Represent every transformation as a single, consistent matrix-vector multiplication:

$$P' = M_{transform} \cdot P$$

Matrices for Scaling and Rotation

We can represent a 2D point $P = (x, y)$ as a column vector $\begin{pmatrix} x \\ y \end{pmatrix}$.

Scaling Matrix:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}}_{M_{scale}} \begin{pmatrix} x \\ y \end{pmatrix}$$

Matrices for Scaling and Rotation

We can represent a 2D point $P = (x, y)$ as a column vector $\begin{pmatrix} x \\ y \end{pmatrix}$.

Scaling Matrix:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}}_{M_{scale}} \begin{pmatrix} x \\ y \end{pmatrix}$$

Rotation Matrix:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}}_{M_{rotate}} \begin{pmatrix} x \\ y \end{pmatrix}$$

Matrices for Scaling and Rotation

We can represent a 2D point $P = (x, y)$ as a column vector $\begin{pmatrix} x \\ y \end{pmatrix}$.

Scaling Matrix:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}}_{M_{scale}} \begin{pmatrix} x \\ y \end{pmatrix}$$

Rotation Matrix:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}}_{M_{rotate}} \begin{pmatrix} x \\ y \end{pmatrix}$$

But what about Translation?

There is no 2x2 matrix that can perform addition. Matrix multiplication is a *linear* transformation (origin stays fixed), but translation is an *affine* one (origin moves).

Homogeneous Coordinates & The w Component

Different w values = Different Scales of the Same Space

The Concept of w

In homogeneous coordinates, w acts as a **scale factor**.

- The "Real World" is the slice where $w = 1$.
- Changing w scales the coordinate values, but represents the *same* location in space (once normalised).

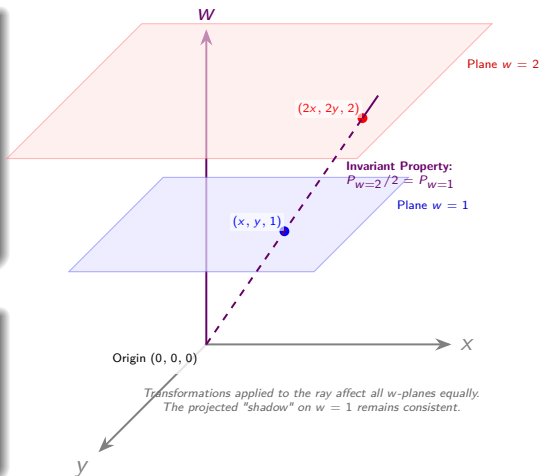
Preservation of Position

Any point (x, y, w) is equivalent to $(x/w, y/w, 1)$.

$$(2, 4, 2) \equiv (1, 2, 1)$$

They lie on the same **Projective Ray**.

"Think of higher w planes as larger, scaled versions of the same universe."



The Solution: Homogeneous Coordinates

A clever trick to make translation a multiplication.

The Idea: We add an extra dimension, the 'w' coordinate, and set it to 1.

- Our 2D point (x, y) becomes a 3D vector $\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$.
- Our transformation matrices become 3x3.

The Solution: Homogeneous Coordinates

A clever trick to make translation a multiplication.

The Idea: We add an extra dimension, the 'w' coordinate, and set it to 1.

- Our 2D point (x, y) becomes a 3D vector $\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$.
- Our transformation matrices become 3x3.

By doing this, we can use the extra column in the matrix to encode translation!

Translation Matrix (Homogeneous):

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}$$

The Solution: Homogeneous Coordinates

A clever trick to make translation a multiplication.

The Idea: We add an extra dimension, the 'w' coordinate, and set it to 1.

- Our 2D point (x, y) becomes a 3D vector $\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$.
- Our transformation matrices become 3x3.

By doing this, we can use the extra column in the matrix to encode translation!

Translation Matrix (Homogeneous):

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}$$

This is a breakthrough! All three transformations can now be represented by a 3x3 matrix multiplication.

Updated Matrices (3x3 Homogeneous)

We embed our original 2x2 matrices into a 3x3 identity matrix.

Scaling:

$$M_{scale} = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Updated Matrices (3x3 Homogeneous)

We embed our original 2x2 matrices into a 3x3 identity matrix.

Scaling:

$$M_{scale} = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotation:

$$M_{rotate} = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Updated Matrices (3x3 Homogeneous)

We embed our original 2x2 matrices into a 3x3 identity matrix.

Scaling:

$$M_{scale} = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotation:

$$M_{rotate} = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Translation:

$$M_{translate} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

Why Do We Need a 4th Coordinate?

Solving the "Translation Problem"

You may notice our vectors are $(x, y, z, \mathbf{w})$. Why add a 4th dimension ($w = 1$)?

The Problem (3D)

In 3D, Matrix Multiplication can Rotate or Scale, but it **cannot Move (Translate)** the origin $(0, 0, 0)$.

To move an object, we must **Add** a vector:

$$P' = (R \cdot P) + T$$

Issue: We cannot combine R and T into one matrix. We can't chain operations easily.

The Solution (Homogeneous)

By adding $w = 1$, we turn Translation into Multiplication.

$$P' = M \cdot P$$

Now, Translation, Rotation, and Scale are all just **one matrix** (4×4). We can pre-multiply them into a single **Model Matrix**.

Rotation / Scale Translation

$$\left[\begin{array}{ccc|c} R_{xx} & R_{xy} & R_{xz} & T_x \\ R_{yx} & R_{yy} & R_{yz} & T_y \\ R_{zx} & R_{zy} & R_{zz} & T_z \\ \hline 0 & 0 & 0 & 1 \end{array} \right] \cdot \begin{bmatrix} x \\ y \\ z \\ \mathbf{1} \end{bmatrix} = \begin{bmatrix} (R \cdot \text{pos}) + (T_x \cdot \mathbf{1}) \\ (R \cdot \text{pos}) + (T_y \cdot \mathbf{1}) \\ (R \cdot \text{pos}) + (T_z \cdot \mathbf{1}) \\ 1 \end{bmatrix}$$

$w = 1$

*The 1 multiplies with T ,
adding it to the result!*

3D Transformations Cheat Sheet: Part 1

Translation and Scaling

A quick reference for mapping standard mathematical matrices to their Three.js counterparts.

Operation	Homogeneous Matrix (4x4)	Three.js Implementation	Explanation
Translation	$\begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$	<pre>mesh.position.set(x, y, z); // OR mesh.translateX(x);</pre>	Affine shift. Moves the object's origin by the vector (t_x, t_y, t_z) . The $w = 1$ component allows the right-most column to add to x, y, z .
Scaling	$\begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	<pre>mesh.scale.set(x, y, z); // OR mesh.scale.x = x;</pre>	Resizing. Multiplies each local coordinate by a scalar. If $s_x \neq s_y \neq s_z$, the scale is non-uniform and can distort normals.

3D Transformations Cheat Sheet: Part 2

Rotations across the Primary Axes

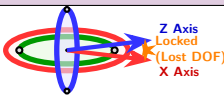
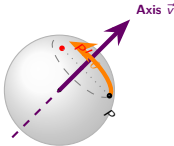
In 3D, rotation matrices change depending on which axis acts as the "skewer" or pivot. Three.js handles this using Euler angles or Quaternions.

Axis	Rotation Matrix (4x4)	Three.js Implementation	Visual Result
X-Axis (Pitch)	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	<pre>mesh.rotation.x = rad; // OR mesh.rotateX(rad);</pre>	The object nods forward or backward. The X coordinates remain completely unchanged.
Y-Axis (Yaw)	$\begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	<pre>mesh.rotation.y = rad; // OR mesh.rotateY(rad);</pre>	The object turns left or right (like a car turning). The Y coordinates remain completely unchanged.
Z-Axis (Roll)	$\begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	<pre>mesh.rotation.z = rad; // OR mesh.rotateZ(rad);</pre>	The object tilts side-to-side (like an airplane doing a barrel roll). The Z coordinates remain completely unchanged.

Rotations: Euler vs. Quaternions

Solving the Gimbal Lock Problem

While standard rotation matrices (Euler angles) are intuitive, they suffer from mathematical flaws when animating complex 3D objects. Quaternions provide the robust solution.

Format	Definition	The Problem / Solution	Visual Concept
Euler Matrix	Applies rotations sequentially around 3 axes. <i>Pros:</i> Intuitive to read.	Problem: Gimbal Lock. Rotating a middle axis by 90° can cause the first and third axes to align. <i>Result:</i> You lose a Degree of Freedom (DOF); animations become erratic.	 Z Axis Locked (Lost DOF) X Axis
Quaternion	A 4D complex number (w, x, y, z) representing a single continuous rotation around an arbitrary 3D axis $\vec{v}$.	Solution: SLERP. Evaluates all axes simultaneously. No Gimbal Lock: Perfect stability. SLERP: Spherical Linear Interpolation ensures smooth, shortest-path animations.	 Axis $\vec{v}$

The Power of Combining Transformations

One Matrix to Rule Them All

The real magic of matrices is that they can be combined.

- A sequence of transformations (e.g., scale, then rotate, then translate) can be pre-multiplied into a single matrix.

$$M_{combined} = M_{translate} \cdot M_{rotate} \cdot M_{scale}$$

- This calculation is done once on the CPU.
- The GPU then only needs to perform **one** matrix-vector multiplication per vertex.

$$P' = M_{combined} \cdot P$$

The Power of Combining Transformations

One Matrix to Rule Them All

The real magic of matrices is that they can be combined.

- A sequence of transformations (e.g., scale, then rotate, then translate) can be pre-multiplied into a single matrix.

$$M_{combined} = M_{translate} \cdot M_{rotate} \cdot M_{scale}$$

- This calculation is done once on the CPU.
- The GPU then only needs to perform **one** matrix-vector multiplication per vertex.

$$P' = M_{combined} \cdot P$$

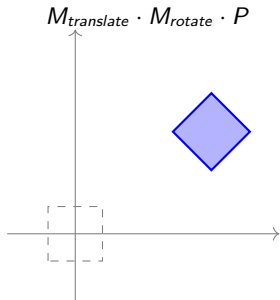
This is incredibly efficient and is the core of the modern graphics pipeline.

Warning: Order Matters!

Matrix Multiplication is Not Commutative ($A \cdot B \neq B \cdot A$)

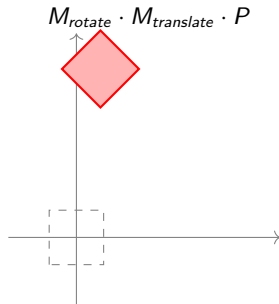
The order you apply transformations has a huge impact. Transformations are applied right-to-left.

1. Rotate, then 2. Translate



This is usually what you want.

1. Translate, then 2. Rotate



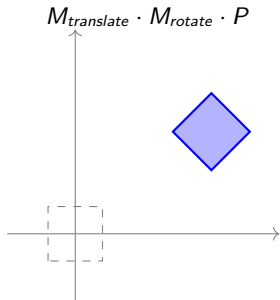
The object "orbits" the origin.

Warning: Order Matters!

Matrix Multiplication is Not Commutative ($A \cdot B \neq B \cdot A$)

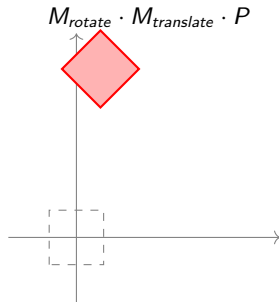
The order you apply transformations has a huge impact. Transformations are applied right-to-left.

1. Rotate, then 2. Translate



This is usually what you want.

1. Translate, then 2. Rotate



The object "orbits" the origin.

The standard convention is **Scale** $\rightarrow$ **Rotate** $\rightarrow$ **Translate** (TRS).

$$M_{\text{final}} = M_{\text{Translate}} \cdot M_{\text{Rotate}} \cdot M_{\text{Scale}}$$

Sequencing: Math vs. The Three.js Abstraction

Do we still need to follow inverse sequencing?

1. The Mathematical Reality

In raw WebGL/Linear Algebra, transformations are **non-commutative**

$(A \cdot B \neq B \cdot A)$.

- Matrices must be multiplied in **inverse order**.
- To Scale $\rightarrow$ Rotate $\rightarrow$ Translate, you must write:

$$M_{final} = T \cdot R \cdot S$$

```
mesh.position.set(5,0,0);  
mesh.rotation.y = 1.5;  
// Order A
```

2. The Three.js Property System

Three.js abstracts the Model Matrix.

- Code statement order **does not matter** for properties like `.position` or `.rotation`.
- `renderer.render()` always composes the matrix in a fixed, safe sequence:

Fixed Order:

Matrix = $T * R * S$

```
mesh.rotation.y = 1.5;  
mesh.position.set(5,0,0);  
// Order B
```

SAME MODEL MATRIX

The Exception

Statement order **only** matters if you are using relative methods like `mesh.rotateX()` or `mesh.translateX()`, or if you manually set `matrixAutoUpdate = false`.

Animation: Bringing it to Life

The Golden Rule: Frame Rate Independence

To create motion, we update transformation values inside the **render loop**. However, how we calculate that update determines if our game is playable or broken.

The Naive Way (Frame-Based)

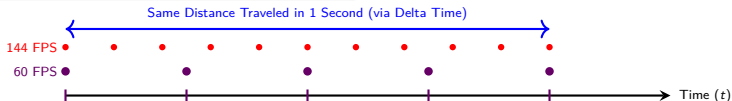
```
position += speed;
```

The Trap: On a 144Hz monitor, the object moves **2.4x faster** than on a 60Hz monitor. Speed becomes tied to hardware!

The Pro Way (Time-Based)

```
position += speed * deltaTime;
```

The Win: Movement is calculated in *meters per second*, not *pixels per frame*. Consistent on all PCs.



Implementation: The Delta Time Pattern

```
let lastTime = 0;
function render(currentTime) {
  currentTime *= 0.001; // ms to seconds
  const deltaTime = currentTime - lastTime;
  lastTime = currentTime;

  rotation += rotationSpeed * deltaTime; // Smooth & Consistent
  requestAnimationFrame(render);
}
```


The Journey of a Vertex

From Model to Screen

Rendering a 3D model is a journey through several coordinate systems, orchestrated by a sequence of three critical matrices.



The Journey of a Vertex

From Model to Screen

Rendering a 3D model is a journey through several coordinate systems, orchestrated by a sequence of three critical matrices.



The "Movie Set" Analogy:

- 1. Model Matrix** *"Placing the actor."* Moving the object from its local origin (0,0) to a specific spot in the world coordinates.
- 2. View Matrix** *"Positioning the camera."* Moving the entire world so that the camera is at the centre (0,0), looking down the Z-axis.
- 3. Projection Matrix** *"Choosing the lens."* Applying perspective (things look smaller further away) and flattening 3D data for the 2D screen.

The Journey of a Vertex

From Model to Screen

Rendering a 3D model is a journey through several coordinate systems, orchestrated by a sequence of three critical matrices.



The "Movie Set" Analogy:

- 1. Model Matrix** *"Placing the actor."* Moving the object from its local origin (0,0) to a specific spot in the world coordinates.
- 2. View Matrix** *"Positioning the camera."* Moving the entire world so that the camera is at the centre (0,0), looking down the Z-axis.
- 3. Projection Matrix** *"Choosing the lens."* Applying perspective (things look smaller further away) and flattening 3D data for the 2D screen.

This sequence is known as the **MVP (Model-View-Projection)** pipeline.

The "M" in MVP: The Model Matrix

Placing the Object in the World

- **Starts in: Model Space.**

- The local coordinate system of the 3D model itself.
- Example: A car model designed with its centre at $(0, 0, 0)$.

The "M" in MVP: The Model Matrix

Placing the Object in the World

- **Starts in: Model Space.**
 - The local coordinate system of the 3D model itself.
 - Example: A car model designed with its centre at (0, 0, 0).
- **The Model Matrix** applies the object's unique scale, rotation, and translation to position it in the scene.

$$M_{Model} = M_{Translate} \cdot M_{Rotate} \cdot M_{Scale}$$

The "M" in MVP: The Model Matrix

Placing the Object in the World

- **Starts in: Model Space.**

- The local coordinate system of the 3D model itself.
- Example: A car model designed with its centre at (0, 0, 0).

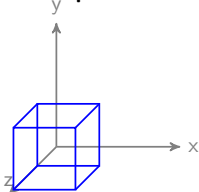
- **The Model Matrix** applies the object's unique scale, rotation, and translation to position it in the scene.

$$M_{Model} = M_{Translate} \cdot M_{Rotate} \cdot M_{Scale}$$

- **Ends in: World Space.**

- The global coordinate system shared by all objects in the scene.

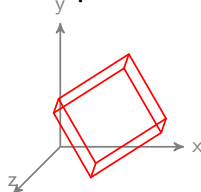
Model Space



Model Matrix



World Space



The "V" in MVP: The View Matrix

Positioning the Camera

- **Starts in: World Space.**
- **The Concept:** Instead of moving the camera, we move the entire world to be in front of a fixed camera.
 - Analogy: Turning a snow globe in your hands instead of walking around it.

The "V" in MVP: The View Matrix

Positioning the Camera

- **Starts in: World Space.**
- **The Concept:** Instead of moving the camera, we move the entire world to be in front of a fixed camera.
 - Analogy: Turning a snow globe in your hands instead of walking around it.
- **The View Matrix** is the inverse of the camera's world transformation. It positions the world so the camera is at the origin, looking down the $-Z$ axis.

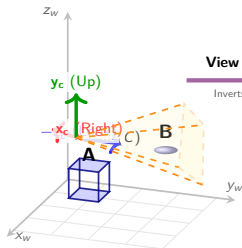
The "V" in MVP: The View Matrix

Positioning the Camera

- **Starts in: World Space.**
- **The Concept:** Instead of moving the camera, we move the entire world to be in front of a fixed camera.
 - Analogy: Turning a snow globe in your hands instead of walking around it.
- **The View Matrix** is the inverse of the camera's world transformation. It positions the world so the camera is at the origin, looking down the $-Z$ axis.
- **Ends in: View Space** (or Camera Space).

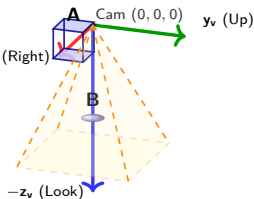
2. View Space

1. World Space



View Matrix (M_{view})

Inverts Camera Transform
 $R_{cam}^T \cdot T_{cam}^{-1}$



The Virtual Camera

Deriving the View Matrix (M_{view})

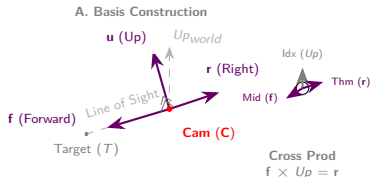
1. The Intuition (Inverse Transformations)

To render from the camera's perspective, we don't move the camera. We move the **World** in the opposite direction.

- Translation: Shift world by $-C$ (Eye position).
- Rotation: Align world axes with camera basis (r, u, f).

2. The "LookAt" Recipe

We construct the Camera Basis vectors using the **Eye (C)**, **Target (T)**, and **World Up ($\vec{U}_{p_{world}}$)**.



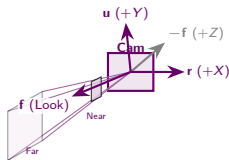
The Gram-Schmidt Process (Right-Handed)

- 1 **Forward (f)**: Vector pointing *towards* the Target.
 $f = \text{normalize}(T - C)$
- 2 **Right (r)**: Perpendicular to Forward and World Up.
 $r = \text{normalize}(f \times \vec{U}_{p_{world}})$
- 3 **True Up (u)**: Perpendicular to Right and Forward.
 $u = \text{normalize}(r \times f)$

3. The Final Matrix

$$M_{view} = \begin{bmatrix} r_x & r_y & r_z & -r \cdot C \\ u_x & u_y & u_z & -u \cdot C \\ f_x & f_y & f_z & f \cdot C \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

B. Camera Space

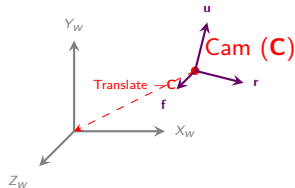


Mathematically Formulating the View Matrix

Converting World Space to Camera Space

The Logic of Inversion

- To view the scene, we must move the camera to $(0, 0, 0)$ and align its axes with the world axes.
- **Step 1 (Translation):** Shift the world by $-C$ (negative camera position) to bring the "eye" to the origin.
- **Step 2 (Rotation):** Align world axes with camera axes using the *inverse* orientation (R^{-1}).
 - Since camera basis vectors (r, u, f) are **orthonormal**, the inverse is simply the **transpose** (R^T).
 - This maps the camera's local axes back to the world's cardinal X, Y, Z directions.



The View Matrix Derivation ($V = R^T \cdot T^{-1}$)

$$V = \underbrace{\begin{pmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ -f_x & -f_y & -f_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{\text{Inverse Rotation } (R^T)} \cdot \underbrace{\begin{pmatrix} 1 & 0 & 0 & -C_x \\ 0 & 1 & 0 & -C_y \\ 0 & 0 & 1 & -C_z \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{\text{Inverse Translation } (T^{-1})} = \begin{pmatrix} r_x & r_y & r_z & -r \cdot C \\ u_x & u_y & u_z & -u \cdot C \\ -f_x & -f_y & -f_z & f \cdot C \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

The "P" in MVP: The Projection Matrix

Creating the Illusion of Depth

- **Starts in: View Space.**
- **The Goal:** To project the 3D scene onto a 2D plane, simulating perspective.

The "P" in MVP: The Projection Matrix

Creating the Illusion of Depth

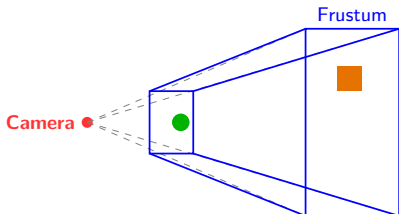
- **Starts in: View Space.**
- **The Goal:** To project the 3D scene onto a 2D plane, simulating perspective.
- **The Projection Matrix** defines a viewing volume (a **frustum**) and squashes it into a perfect cube called **Clip Space**, where X, Y, and Z coordinates all range from -1 to +1.
 - Objects farther away are made smaller.
 - Anything outside the frustum is clipped (discarded).

The "P" in MVP: The Projection Matrix

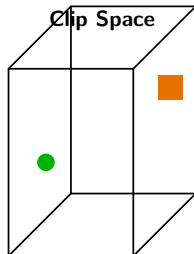
Creating the Illusion of Depth

- **Starts in: View Space.**
- **The Goal:** To project the 3D scene onto a 2D plane, simulating perspective.
- **The Projection Matrix** defines a viewing volume (a **frustum**) and squashes it into a perfect cube called **Clip Space**, where X, Y, and Z coordinates all range from -1 to +1.
 - Objects farther away are made smaller.
 - Anything outside the frustum is clipped (discarded).
- **Ends in: Clip Space.**

View Space



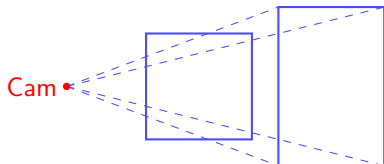
Projection
Matrix



(Normalised Cube
-1 to +1 on all axes)

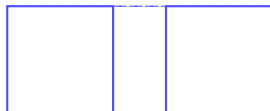
Two Types of Projection

Perspective Projection



- Viewing volume is a **frustum**.
- Objects farther away appear smaller.
- Used in most 3D games and realistic rendering.
- Needs: **FOV**, **Aspect Ratio**, **Near/Far planes**.

Orthographic Projection



- Viewing volume is a rectangular box.
- No perspective; size is constant with distance.
- Used for diagrams, architectural plans, or 2D-style games.

Mathematical Comparison of Projections

Linear Mapping vs. The Perspective Divide

Matrix Formulations (4×4)

$$M_{\text{Orthographic}} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \text{vs.} \quad M_{\text{Perspective}} = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Feature	Orthographic	Perspective
View Volume	Rectangular Box (Cuboid)	Frustum (Truncated Pyramid)
Parallel Lines	Remain parallel	Converge at vanishing points
Matrix Type	Affine (Linear)	Non-Affine (Projective)
The w Effect	w stays 1 (no scaling)	$w' = -z$ (enables <i>Perspective Divide</i>)
Use Case	Engineering, CAD, 2D UI	Realistic 3D Scenes, Games

The "Magic" of Perspective

The bottom row of the Perspective matrix $(0 \quad 0 \quad -1 \quad 0)$ is critical. It moves the z coordinate into the w slot. The hardware then performs:

$$P_{\text{final}} = (x/w, y/w, z/w)$$

This division by distance is what creates the illusion of depth in 3D graphics.

Deconstructing the Projection Matrices

How Scaling, Translation, and Projection Combine

Operation	Orthographic Term	Perspective Term	Functional Purpose
X-Scale	$\frac{2}{r-l}$	$\frac{2n}{r-l}$	Resizes Width to NDC range (2 units)
Y-Scale	$\frac{2}{t-b}$	$\frac{2n}{t-b}$	Resizes Height to NDC range (2 units)
Z-Scale	$\frac{-2}{f-n}$	$-\frac{f+n}{f-n}$	Maps Depth to $[-1, 1]$ range
Translation	$-\frac{r+l}{r-l}, -\frac{t+b}{t-b}$	$\frac{r+l}{r-l}, \frac{t+b}{t-b}$	Centers the volume at the origin (0, 0, 0)
Projection	0	-1	Copies $-Z$ to W for depth division

Orthographic: Linear Scale

$$P_{ortho} = T_{shift} \cdot S_{scale}$$

Purely **Affine**. It treats all points equally regardless of distance. Parallel lines in World Space **never** meet.

Orthographic (Linear)

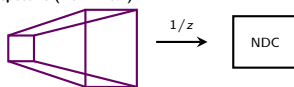


Perspective: Non-Linear

$$P_{persp} = M_{divide} \cdot M_{affine}$$

Non-Affine. The -1 term creates a w that varies with z . This makes the transformation "converge" toward a vanishing point.

Perspective (Non-Linear)



The Final MVP Matrix

One Matrix to Rule Them All

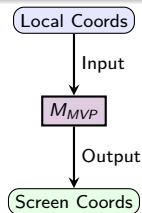
Just as in 2D, we can pre-multiply the three main matrices into a single, all-powerful **Model-View-Projection (MVP) Matrix**.

The MVP Formula

$$M_{MVP} = \underbrace{M_{Projection}}_{\text{Step 3}} \cdot \underbrace{M_{View}}_{\text{Step 2}} \cdot \underbrace{M_{Model}}_{\text{Step 1}}$$

- **Right to Left:** Transformations apply in order from right to left.
- **CPU Pre-calc:** Combined once per object on the CPU to save GPU cycles.
- **GPU Uniform:** Sent as a single variable, making vertex processing lightning fast.

Single GPU Operation!



$$P' = M_{MVP} \cdot P_{local}$$

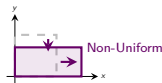
Scene Construction and Modeling Operations

All modeled by Transformation Matrices

Different operations utilise specific parts of the 4×4 transformation matrix.

Modeling Operations	Primary Transformations	Matrix Form Components	Technical Constraint
1. Object Modeling	Scaling (S) <i>Sizing & Shaping</i>	$\begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	Uniform Only? Non-uniform S distorts normals. Requires $(M^{-1})^T$ fix.
2. Animation & Scene	Rigid Body ($R + T$) <i>Placement & Rotation</i>	$\begin{bmatrix} R_{xx} & R_{xy} & R_{xz} & T_x \\ R_{yx} & R_{yy} & R_{yz} & T_y \\ R_{zx} & R_{zy} & R_{zz} & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$	Safe for Normals. Orthogonal rotation preserves angles. No inverse transpose needed.
3. Deformation	Shear (H) <i>Squash, Stretch, Slant</i>	$\begin{bmatrix} 1 & H_{xy} & H_{xz} & 0 \\ H_{yx} & 1 & H_{yz} & 0 \\ H_{zx} & H_{zy} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	Destructive. Alters internal geometry angles. Normals must be recalculated.

Modeling (Scaling)



Scene (Rigid Body)



Deformation (Shear)



The Golden Rule of Matrices

If your matrix M contains **only** Rotation and Translation, you can transform Normals with M . If it contains Scale or Shear, you **MUST** use $(M^{-1})^T$.

Virtual Camera and View Frustum Projection

Formulating Scene Control as Matrices

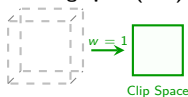
The "Scene Control" is handled by two matrices that simulate a physical camera and its lens.

Pipeline Stage	Primary Function	Matrix Form Components	Technical Constraint
1. Virtual Camera	View Matrix (V) <i>World Inversion</i>	$\begin{bmatrix} R_{xx}^T & R_{xy}^T & R_{xz}^T & -(R^T T)_x \\ R_{yx}^T & R_{yy}^T & R_{yz}^T & -(R^T T)_y \\ R_{zx}^T & R_{zy}^T & R_{zz}^T & -(R^T T)_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$	Inverse Operation. The camera never actually moves. Moves the entire world in the opposite direction.
2. Orthographic	Linear Projection <i>Parallel / No Depth</i>	$\begin{bmatrix} S_x & 0 & 0 & T_x \\ 0 & S_y & 0 & T_y \\ 0 & 0 & S_z & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$	Affine Mapping. Parallel lines remain parallel. The w component stays 1 (no division).
3. Perspective	Frustum Projection <i>Realistic Depth Cues</i>	$\begin{bmatrix} S_x & 0 & A & 0 \\ 0 & S_y & B & 0 \\ 0 & 0 & C & D \\ 0 & 0 & -1 & 0 \end{bmatrix}$	Non-Affine. Objects get smaller with distance. Bottom row triggers division by z .

View Matrix (Inversion)



Orthographic (Box)



Perspective (Frustum)

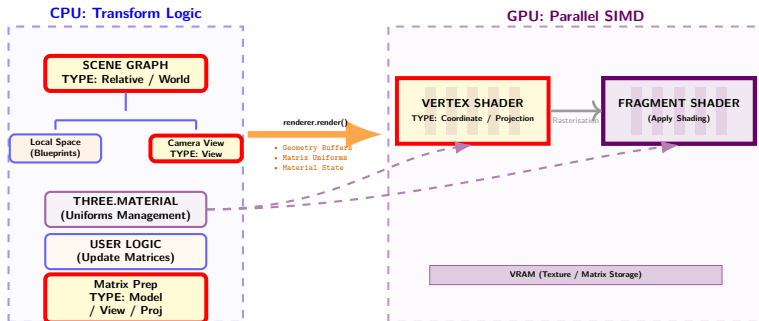


The "Magic" of Perspective Division

The -1 in the bottom row of the Perspective matrix converts 3D View Space into the unit cube. It injects z into the w component. The graphics hardware then automatically divides x , y , and z by w , scaling distant objects down and creating the illusion of depth.

Transformations in the Three.js Pipeline

Identifying Transformation Types & Responsibilities



Transformation Responsibilities

- **Scene Graph:** Updates Model Matrices to place blueprints in World Space.
- **Camera View:** Defines the View Transformation relative to the camera lens.
- **Matrix Prep:** CPU combines matrices into 4x4 formats for efficient GPU handoff.
- **Vertex Shader:** Massively parallel SIMD math transforms vertices into Clip Space.

The MVP Matrix in Practice

The Core of a Modern Vertex Shader

The combined MVP matrix makes the vertex shader's job beautifully simple. Its only task is to perform a single multiplication for each vertex.

GLSL Vertex Shader Code

```
// A 'uniform' is data sent from the CPU to the GPU
// that is the same for all vertices in a draw call.
uniform mat4 u_mvMatrix;

// An 'in' variable (or 'attribute') is per-vertex data,
// like the position, that comes from a vertex buffer.
in vec4 a_position;

void main() {
    // Transform the vertex's model-space position into
    // final clip-space position and assign it to the
    // special gl_Position output variable.
    gl_Position = u_mvMatrix * a_position;
}
```

Practical Example: A three.js Scene

How a Library Abstracts These Concepts

High-level libraries like 'three.js' handle the matrix math for you, but the MVP concepts are still there under the hood.

```
// Scene holds all objects, lights, and cameras
const scene = new THREE.Scene();

// Camera defines the VIEW and PROJECTION
// PerspectiveCamera(fov, aspect, near, far)
const camera = new THREE.PerspectiveCamera(75, w/h, 0.1, 1000);

// An object is a "Mesh" (Geometry + Material)
const geometry = new THREE.BoxGeometry(1, 1, 1);
const material = new THREE.MeshNormalMaterial();
const cube = new THREE.Mesh(geometry, material);
scene.add(cube);
```

Listing 1: Scene

The MVP in three.js

Manipulating Transforms with Properties

The Model Matrix is controlled by the object's properties:

```
// These properties are used to build the MODEL matrix
cube.scale.set(1.5, 1, 1);           // S
cube.rotation.y = Math.PI / 4;       // R
cube.position.x = 2;                  // T
```


The MVP in three.js

Manipulating Transforms with Properties

The Model Matrix is controlled by the object's properties:

```
// These properties are used to build the MODEL matrix
cube.scale.set(1.5, 1, 1);           // S
cube.rotation.y = Math.PI / 4;       // R
cube.position.x = 2;                  // T
```

The View Matrix is the inverse of the camera's position/orientation:

```
// These properties are used to build the VIEW matrix
camera.position.set(1, 2, 5);
camera.lookAt(cube.position); // Point camera at the cube
```

The MVP in three.js

Manipulating Transforms with Properties

The Model Matrix is controlled by the object's properties:

```
// These properties are used to build the MODEL matrix
cube.scale.set(1.5, 1, 1);           // S
cube.rotation.y = Math.PI / 4;       // R
cube.position.x = 2;                  // T
```

The View Matrix is the inverse of the camera's position/orientation:

```
// These properties are used to build the VIEW matrix
camera.position.set(1, 2, 5);
camera.lookAt(cube.position); // Point camera at the cube
```

The Projection Matrix was already defined when we created the 'PerspectiveCamera'.

The Render Loop: Putting it all Together

What 'renderer.render()' really does

The 'animate' function is called every frame.

```
function animate() {  
    // This one line does all the work!  
    renderer.render(scene, camera);  
  
    requestAnimationFrame(animate);  
}
```

The Render Loop: Putting it all Together

What 'renderer.render()' really does

The 'animate' function is called every frame.

```
function animate() {  
    // This one line does all the work!  
    renderer.render(scene, camera);  
  
    requestAnimationFrame(animate);  
}
```

When you call 'renderer.render(scene, camera)', 'three.js' performs these steps:

- 1 It checks 'cube.position', 'rotation', 'scale' and computes its final **Model** matrix.
- 2 It checks 'camera.position' and computes the **View** matrix.
- 3 It uses the camera's settings (FOV, etc.) to get the **Projection** matrix.
- 4 It computes the final **MVP** matrix ($P \cdot V \cdot M$).
- 5 It sends the MVP matrix and vertex data to the GPU and tells it to draw.

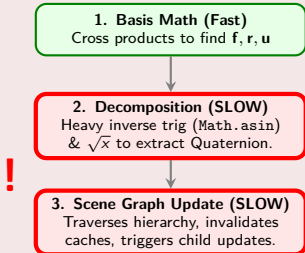
Caution: Under the Hood of `lookAt()`

Why a simple function can destroy your frame rate

The Misconception: `lookAt()` just rotates an object.

The Reality: It completely destroys and rebuilds the local coordinate system.

The "three.js" Overhead Pipeline



Why not a simple inverse matrix?

A basic inverse matrix only creates a **View Matrix** for the whole world.

Because three.js uses a **Scene Graph**, `lookAt()` must compute the target's absolute world position, calculate the orientation, and then mathematically *reverse-engineer* it through all parent transforms to update the object's **local** .`quaternion`.

Best Practices for High Performance

If handling hundreds of objects (e.g., crowds, particles), avoid calling `lookAt()` in the `animate()` loop!

1. Static Objects (Init Only)

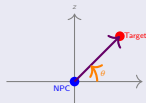
Call `lookAt()` *once* during setup. Never in the loop.

2. Dynamic Crowds (Use the GPU)

Pass target positions as a uniform and do the math in a **Vertex Shader**.

3. The 2D Bypass (`Math.atan2`)

If NPCs only turn left/right on a flat floor, bypass matrices entirely:



```
rot.y = Math.atan2(dx, dz);
```

The "Matrix" Revelation

Seeing the World in Numbers

Recall our "Frozen World" problem. How do we make it live?

"There is no spoon. There is only the Matrix."

The "Matrix" Revelation

Seeing the World in Numbers

Recall our "Frozen World" problem. How do we make it live?

"There is no spoon. There is only the Matrix."

Every frame of your favorite video game is just a conversation between these three matrices:

- **The Mouse Look:** You are not turning a head. You are mathematically *moving the entire universe* around you by updating the **View Matrix**.
- **The Character Animation:** Bones do not exist. A character is just a hierarchy of **Model Matrices** passing rotations from shoulder, to elbow, to wrist.
- **The Sniper Zoom:** You are not changing lenses. You are simply narrowing the *FOV* angle inside the **Projection Matrix** to stretch the image.



The "Matrix" Revelation

Seeing the World in Numbers

Recall our "Frozen World" problem. How do we make it live?

"There is no spoon. There is only the Matrix."

Every frame of your favorite video game is just a conversation between these three matrices:

- **The Mouse Look:** You are not turning a head. You are mathematically *moving the entire universe* around you by updating the **View Matrix**.
- **The Character Animation:** Bones do not exist. A character is just a hierarchy of **Model Matrices** passing rotations from shoulder, to elbow, to wrist.
- **The Sniper Zoom:** You are not changing lenses. You are simply narrowing the *FOV* angle inside the **Projection Matrix** to stretch the image.



The Takeaway

Linear algebra is not just abstract math. It is the engine that drives every single frame of every 3D experience you have ever had.

Constructing & Animating a 3D Model

Case Study: The Hierarchical Lego Car

1. Component Assembly (Local Space)

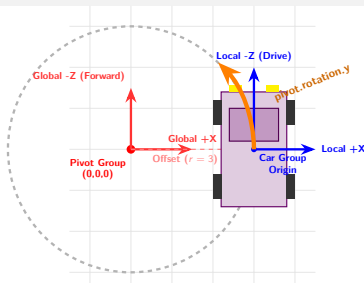
- Parts (cab, wheels) are created at the origin, then **Translated** and **Rotated** into position relative to the chassis.

2. The Scene Graph (Grouping)

- By adding all parts to a `THREE.Group`, we bind their transformation matrices. Moving the group moves the entire car as one unified object.

3. Pivot Animation (Global Space)

- To make the car drive in a circle, we do *not* calculate complex sine/cosine paths. We offset the car from a **Pivot Group** and simply rotate the pivot!



Three.js Implementation

```
// 1. Group the car parts together
const carGroup = new THREE.Group();
carGroup.add(chassis, cab, wheels);

// 2. Offset car and attach to pivot
carGroup.translateX(3); // Radius of turn
const pivotGroup = new THREE.Group();
pivotGroup.add(carGroup);
scene.add(pivotGroup);

// 3. Animate: Rotate the pivot!
function animate() {
    pivotGroup.rotation.y += 0.05;
    requestAnimationFrame(animate);
}
```

Procedural Terrain & Object Placement

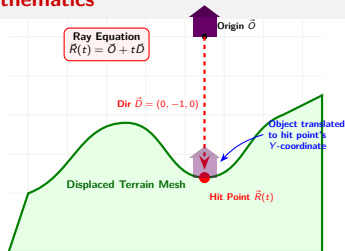
Case Study: Vertex Displacement and Raycasting Mathematics

1. Modeling the Hilly Landscape

- We start with a highly subdivided 2D flat plane.
- **Vertex Displacement:** We iterate through every vertex and alter its height (Y) using mathematical functions like sine/cosine or Perlin noise: $y = \sin(x) \times \cos(z)$.

2. The Math of Raycasting

- To place an object (like a house) perfectly on the hills, we shoot a mathematical ray straight down from the sky.
- **Ray Equation:** $\vec{R}(t) = \vec{O} + t\vec{D}$
- $\vec{O}$ is the Origin (object's sky position), $\vec{D}$ is Direction $(0, -1, 0)$, and t is the distance travelled until it hits the mesh.



Three.js Implementation

```
// 1. Create Hills (Displace Z/Y)
const pos = plane.attributes.position;
for(let i=0; i < pos.count; i++) {
    let z = Math.sin(pos.getX(i)*0.3) * 1.5;
    pos.setZ(i, z);
}

// 2. Raycast to drop object
raycaster.set(
    new THREE.Vector3(x, 10, z), // Origin (High Up)
    new THREE.Vector3(0, -1, 0) // Direction (Down)
);
const hits = raycaster.intersectObject(terrain);
if (hits.length > 0) {
    house.position.y = hits[0].point.y; // Snap!
}
```

Summary: Key Takeaways

- **Transformations** (Translate, Rotate, Scale) are essential for dynamic scenes and are best performed on the GPU.

Summary: Key Takeaways

- **Transformations** (Translate, Rotate, Scale) are essential for dynamic scenes and are best performed on the GPU.
- **Matrices** provide a unified way to represent all transformations as a single multiplication.

Summary: Key Takeaways

- **Transformations** (Translate, Rotate, Scale) are essential for dynamic scenes and are best performed on the GPU.
- **Matrices** provide a unified way to represent all transformations as a single multiplication.
- **Homogeneous Coordinates** (the 'w' component) are a mathematical trick that allows translation to be included in this matrix framework.

Summary: Key Takeaways

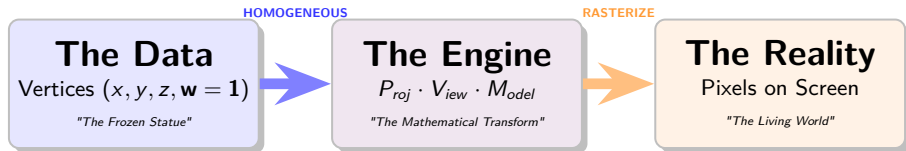
- **Transformations** (Translate, Rotate, Scale) are essential for dynamic scenes and are best performed on the GPU.
- **Matrices** provide a unified way to represent all transformations as a single multiplication.
- **Homogeneous Coordinates** (the 'w' component) are a mathematical trick that allows translation to be included in this matrix framework.
- The **order** of matrix multiplication is critical ($T \cdot R \cdot S$ is standard).

Summary: Key Takeaways

- **Transformations** (Translate, Rotate, Scale) are essential for dynamic scenes and are best performed on the GPU.
- **Matrices** provide a unified way to represent all transformations as a single multiplication.
- **Homogeneous Coordinates** (the 'w' component) are a mathematical trick that allows translation to be included in this matrix framework.
- The **order** of matrix multiplication is critical ($T \cdot R \cdot S$ is standard).
- The **MVP (Model-View-Projection) pipeline** is the standard process for taking a 3D model's vertices and mapping them onto your 2D screen.

The Grand Takeaway

You are not just coding; you are engineering reality.



The "Matrix" Translator

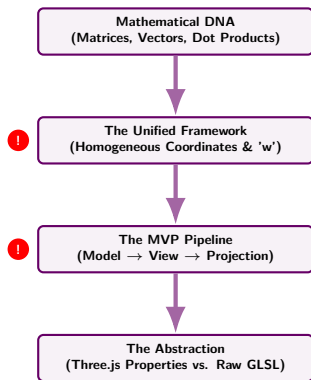
- **To Move a Character?** You don't redraw it. You update the **Model Matrix**.
- **To Move the Camera?** You don't move the viewer. You move the universe inversely via the **View Matrix**.
- **To Zoom In?** You don't change the lens. You narrow the FOV in the **Projection Matrix**.

Linear Algebra is the hidden engine of every frame you see.

Study Guide: Mastering Transformations

A Roadmap for Navigating Linear Algebra and Graphics

1. The Technical Roadmap



Priority 1: Space Awareness

Focus on the **Coordinate Spaces**. Do not just memorise matrices; understand **where** the vertex is at each step of the MVP journey.

2. How to Approach the Topics

- **Think "Right-to-Left"**: Matrix multiplication order is the most common student error. Always read your transform chains from the vertex outward.
- **Visualise the 'w'**: Don't treat Homogeneous coordinates as just a "trick". Understand how the extra dimension enables translation and perspective.
- **Bridge Math to Code**: When using Three.js, mentally map properties like `.position` back to the **Model Matrix**.

Final Advice

If you understand the **MVP Pipeline**, you understand the lecture. It is the "Grand Unification" of everything we have covered today.

Appendix: Advanced Transformations I

Reflection (Mirroring)

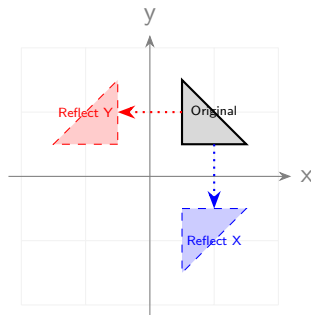
Reflection is a special case of scaling where the scale factor is **-1**. It "flips" the object across an axis.

1. **Reflection across X-axis** Keep x , negate y .

$$M_{reflectX} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

2. **Reflection across Y-axis** Negate x , keep y .

$$M_{reflectY} = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$



Note

Reflection matrices have a determinant of -1 . This "turns the object inside out" (reverses vertex winding order), which can affect backface culling!

Appendix: Advanced Transformations II

Shearing (Skewing)

Shearing "slants" the shape of an object. It shifts one coordinate based on the value of the other.

3. Shear X (Horizontal Skew) Push x to the right as y increases.

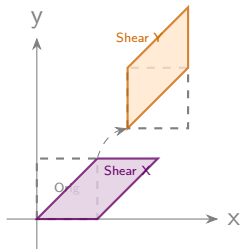
$$x' = x + ky \quad , \quad y' = y$$

$$M_{\text{shearX}} = \begin{pmatrix} 1 & k & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

4. Shear Y (Vertical Skew) Push y up as x increases.

$$x' = x \quad , \quad y' = y + kx$$

$$M_{\text{shearY}} = \begin{pmatrix} 1 & 0 & 0 \\ k & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$



Note: Shearing is often used for creating italic text or simulated shadow projection.

Appendix: Coding the Pipeline (Setup vs. Loop)

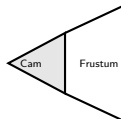
Comparative Overview of Transformations in Three.js

Case	Matrix Components	Code Context	Execution Frequency
1. Setup	Projection (P), View (V)	init() function	Static (Runs Once)
2. Loop	Model (M)	animate() loop	Dynamic (Every Frame)

Case 1: Defining the Lens & Camera

```
// 1. PROJECTION (Lens)
// Static: Defined once at start
const camera = new THREE.PerspectiveCamera(75, w/h,
    0.1, 1000);

// 2. VIEW (Observer)
// Static: Moves world relative to eye
camera.position.set(0, 5, 10);
camera.lookAt(0, 0, 0);
```

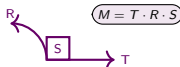


Case 2: Animating the Object

```
function animate(time) {
    const delta = clock.getDelta();

    // 3. MODEL (The Actor)
    // Dynamic: Updated every frame (TRS)
    cube.scale.set(2, 2, 2); // S
    cube.rotation.y += 1.0 * delta; // R
    cube.position.x = Math.sin(time); // T

    renderer.render(scene, camera);
}
```



Appendix: Virtual Camera & Projection

Formulating Virtual Camera and Clipping Volume as Matrices

The "Scene Control" is handled by two matrices that simulate a physical camera.

1. The Virtual Camera (View Matrix)

Concept: The camera never actually moves. Instead, the **View Matrix** (V) moves the *entire world* in the opposite direction.

Formulation: If the Camera's position in the world is defined by a matrix C (Rotation R , Translation T):

$$C = \begin{bmatrix} R & T \\ 0 & 1 \end{bmatrix}$$

The **View Matrix** is the **Inverse** of C :

$$V = C^{-1} = \begin{bmatrix} R^T & -(R^T T) \\ 0 & 1 \end{bmatrix}$$

- **Inverse Translation:** If Cam moves +10, Scene moves -10.
- **Transpose Rotation:** If Cam looks Left, Scene rotates Right.

2. The Projection Matrix (The Lens)

Concept: Converts 3D View Space into the unit cube (Clip Space).

Type A: Orthographic (Architecture)

- Parallel lines remain parallel. No depth cues.
- **Matrix:** simple scaling factors ($1/\text{width}, 1/\text{height}$).
- **Bottom Row:** $[0, 0, 0, 1]$ (No perspective divide).

Type B: Perspective (Human Eye)

- Objects get smaller with distance (z).
- **Matrix:** injects z into the w component.
- **Bottom Row:** $[0, 0, -1, 0]$ (Triggers division by z).



Ortho (Box)



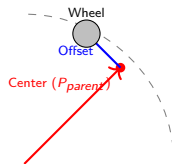
Persp (Frustum)

Appendix: The Challenge of Composite Objects

When T, R, S isn't enough

Real-world objects (like a car) are composed of multiple parts. Without a scene graph (hierarchy), managing them is difficult.

- **The "Flat" World Problem:** The GPU sees a list of independent meshes (Chassis, Wheel_FL, etc.).
- **The Coordination Nightmare:** To turn the car, you cannot just rotate the wheels.
 - The wheels must **orbit** the center.
 - The wheels must **spin** locally.
 - The wheels must **yaw** globally.



To move the wheel, we calculate:
 $P_{wheel} = P_{parent} + (R_{parent} \cdot \text{Offset})$

Appendix: Composition without a Scene Graph

The "Virtual Hierarchy" Approach

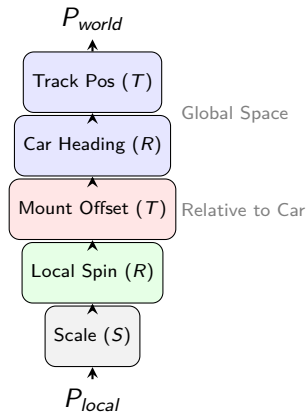
Without a Scene Graph (which we will learn later), we must manually construct a specific chain of matrices for **every single component** (wheels, chassis, cabin) to make them move together.

The Formula: For a single wheel, we calculate a final matrix M every frame:

$$M_{final} = T_{world} \cdot R_{heading} \cdot T_{local} \cdot R_{spin} \cdot S_{scale}$$

- **Right-to-Left Application:**

- 1 **Scale (S)**: Shape the unit cylinder.
- 2 **Local Rotation (R_{spin})**: Spin around the axle.
- 3 **Local Translation (T_{local})**: Move to the "mounting point" (relative to car center).
- 4 **Global Transform ($T_{world} \cdot R_{heading}$)**: Move the entire car to the track position.

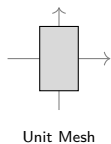


Appendix: Visualising the Matrix Chain

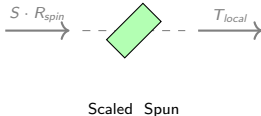
Why Order Matters: The Journey of a Vertex

If we change the order (e.g., apply *Heading* before *Local Offset*), the wheel would orbit the track origin instead of the car!

1. Local Space



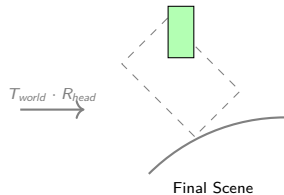
2. Shaped



3. Car Space



4. World Space



The Takeaway

Without a scene graph, you are responsible for calculating this entire chain for every object, every frame. This is error-prone and hard to scale.

Appendix: Mathematical "Grouping"

Using Quaternions to Simulate Hierarchy

To solve this without a physics engine, we use **Quaternions** to mathematically "glue" components together.

1. Grouping Rotations

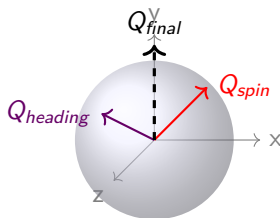
We multiply quaternions to combine local spin with global heading.

$$Q_{final} = \underbrace{Q_{car_heading}}_{\text{Parent}} \cdot \underbrace{Q_{wheel_spin}}_{\text{Child}}$$

2. Grouping Positions

We rotate the local offset by the parent's orientation.

$$P_{world} = P_{car} + (Q_{car_heading} \cdot P_{offset})$$



Multiplication combines rotations robustly

Why not Rotation Matrices?

Multiplying matrices accumulates floating-point errors and is computationally expensive. Quaternions are more stable and efficient for chaining transformations.

Appendix: What is a Quaternion?

A "Magic" 4D Number for Rotation

The Concept: Instead of rotating around X, then Y, then Z (which causes issues like Gimbal Lock), imagine sticking a skewer through an object and spinning it once.

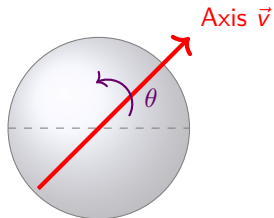
- The **Skewer** is a 3D vector (x, y, z) .
- The **Spin** is an angle θ .

The Math (Axis-Angle Formula): To convert this idea into a Quaternion (w, x, y, z) , we use trigonometry:

Constructing a Quaternion

If axis is normalised $\vec{v} = (v_x, v_y, v_z)$:
 $w = \cos(\frac{\theta}{2})$; $x = v_x \cdot \sin(\frac{\theta}{2})$; $y = v_y \cdot \sin(\frac{\theta}{2})$;
 $z = v_z \cdot \sin(\frac{\theta}{2})$

Why $\theta/2$? It is a complex mathematical requirement (related to "double cover" of rotation groups) that ensures 360° rotations behave correctly.



$$q = (w, \mathbf{v} \sin \frac{\theta}{2})$$

"1 Real part + 3 Imaginary parts"